22. Construct a C Program to Implement Best Fit Algorithm of Memory Management

---

Code - Part 1 (Lines 1-34) -- Memory Initialization & Display

```
1  #include <stdio.h>
2
3  #define MAX_MEMORY 1000
4
5  int memory[MAX_MEMORY];
6
7  // Initialize memory
8  void initializeMemory() {
9      for (int i = 0; i < MAX_MEMORY; i++) {
10         memory[i] = -1;
11     }
12 }
13
14 // Display memory
15 void displayMemory() {
16     int i, j;
17
18     printf("\nMemory Status:\n");
19
20     for (i = 0; i < MAX_MEMORY; i++) {
21         if (memory[i] == -1) {
22             j = i;
23
24             while (j < MAX_MEMORY && memory[j] == -1) {
25                 j++;
26             }
27
28             printf("Free memory block %d-%d\n", i, j - 1);
29             i = j - 1;
30         }
31     }
32 }
33
34 // Best-Fit Allocation
```

Code - Part 2 (Lines 35-67) -- Best-Fit Allocation Logic

```
35 void allocateMemory(int processId, int size) {
36
37     int bestStart = -1;
38     int bestSize = MAX_MEMORY + 1;
39
40     int i = 0;
41
42     while (i < MAX_MEMORY) {
43         if (memory[i] == -1) {
44
45             int start = i;
46             int count = 0;
47
48             while (i < MAX_MEMORY && memory[i] == -1) {
49                 count++;
50                 i++;
51             }
52
53             if (count >= size && count < bestSize) {
54                 bestSize = count;
55                 bestStart = start;
56             }
57
58         } else {
59             i++;
60         }
61     }
62
63     if (bestStart != -1) {
64         for (int i = bestStart; i < bestStart + size; i++) {
65             memory[i] = processId;
66         }
67
```

Code - Part 3 (Lines 68-97) -- Deallocation & Main Function

```
68          printf("Allocated memory block %d-%d to Process %d\n",
69                  bestStart, bestStart + size - 1, processId);
70      } else {
71          printf("Memory allocation for Process %d failed\n", processId);
72      }
73  }
74
75  // Deallocate
76  void deallocateMemory(int processId) {
77      for (int i = 0; i < MAX_MEMORY; i++) {
78          if (memory[i] == processId) {
79              memory[i] = -1;
80          }
81      }
82      printf("Memory released by Process %d\n", processId);
83  }
84
85  int main() {
86
87      initializeMemory();
88      displayMemory();
89
90      allocateMemory(1, 199);
91      displayMemory();
92
93      allocateMemory(2, 100);
94      displayMemory();
95
96      deallocateMemory(1);
97      displayMemory();
98
```

Code - Part 4 (Lines 98-103) -- Main Function (Continued)

```
98
99      allocateMemory(3, 100);
100     displayMemory();
101
102     return 0;
103 }
```

Output

```
Memory Status:
Free memory block 0-999
Allocated memory block 0-198 to Process 1

Memory Status:
Free memory block 199-999
Allocated memory block 199-298 to Process 2

Memory Status:
Free memory block 299-999
Memory released by Process 1

Memory Status:
Free memory block 0-198
Free memory block 299-999
Allocated memory block 0-99 to Process 3

Memory Status:
Free memory block 100-198
Free memory block 299-999


...Program finished with exit code 0
Press ENTER to exit console.
```